Blocking the First Connection: Where the Time Goes in DPI-Guided XDP Enforcement

Shelender Kumar
School of Electrical Engineering and Computer Science (SEECS), NUST
Islamabad, Pakistan

[Co-authors to be confirmed: supervisor, FYP teammates]
SEECS, NUST
Islamabad, Pakistan

ABSTRACT

Gateways that block domains must turn names seen by deep packet inspection (DPI) into packet drops on IP addresses. We ask how long that takes and whether the first connection to a blocked domain gets through. We built DPI Guard, which couples NFStream/nDPI flow inspection with an eBPF/XDP blocklist, and measured three decision strategies on a Linux testbed, 100 trials each. Acting when a flow is exported blocked domains a median of 5,940 ms after the ClientHello (over the 93 of 100 flows it acted on; on a quiet link the rest never were) and never stopped the first connection. Acting on every packet inside the DPI engine cut the median to 536 ms, a delay we trace to buffered packet capture rather than to classification; across emulated round-trip times of 5 to 1,000 ms it stopped only the fraction of first connections that this delay predicts, 12% at 100 ms. A fast path that parses only TLS ClientHellos from an unbuffered, BPF-filtered socket blocked domains 0.44 ms after the ClientHello and stopped every first connection at every round-trip time. When the server name spans two TCP segments, falling back to the per-packet path instead of flow export cuts enforcement from about 6 s to half a second. The XDP hash-map blocklist kept a constant per-packet cost from 1 to 10,000 entries, while linear iptables rules lost 98% of their throughput. We release the system, the testbed and all raw measurements.

KEYWORDS

eBPF, XDP, deep packet inspection, TLS SNI, packet filtering, measurement

1 INTRODUCTION

Network operators block domains, but packets carry IP addresses. A campus, enterprise or home gateway that must stop traffic to `youtube.com` has to learn which server addresses currently serve that name, and then drop their packets fast enough for the policy to hold.

Deep packet inspection solves the first half: libraries such as nDPI [6] read the server name from the TLS ClientHello's Server Name Indication (SNI) [1, 16], from QUIC [11], DNS or the HTTP `Host` header. eBPF and the eXpress Data Path (XDP) [10] solve the second: an XDP program drops packets in the driver, before the kernel allocates socket buffers, and its blocklist can be changed at run time through BPF maps.

This paper studies what happens *between* the two halves: how long it takes from the moment a blocked name appears on the wire until the kernel drops that server's packets, and whether the first connection completes in the meantime. A blocker that lets every first connection through is weak in practice: a page load, an API call or a software update may need only one connection.

We built DPI Guard, an open-source blocker that pairs DPI with XDP [12], and measured three decision strategies. Our main finding: the dominant delay is not DPI or the kernel but how packets reach user space:

- Acting when a flow is *exported* waits for the flow's idle timeout, so the first connection always completes (median time-to-block 5,940 ms over the flows that were enforced); on a quiet link the flow may never be exported.
- Acting in a *per-packet* DPI hook is faster, but buffered libpcap capture delivers packets in blocks on a timer, adding a uniform 0–1 s delay (median 536 ms). At an emulated 100 ms round-trip time it stops only 12% of first connections.
- A *fast path*, a filtered `AF_PACKET` socket that wakes only for TLS ClientHellos, reduces time-to-block to 0.44 ms; the server's first reply is dropped and every first connection fails, at every round-trip time we tested.

Contributions. (1) The design and implementation of a DPI-guided XDP domain blocker with a two-path architecture and a header-free user-space controller that drives BPF maps through the `bpf(2)` system call (§3). (2) A measurement of time-to-block for three decision strategies that identifies capture buffering as the dominant delay, and shows how it decides whether the first connection survives across round-trip times (§5.1, §5.2). (3) A fallback design for ClientHellos the fast path cannot read, measured against the original (§5.3). (4) A comparison of drop cost and blocklist scaling for XDP hash maps, nftables sets and iptables rules (§5.4). (5) A policy model that maps domain rules to IP addresses with exact unblocking and reports collateral blocking (§3).

2 BACKGROUND AND RELATED WORK

Packet capture. User-space monitors read packets through libpcap, which descends from the BSD Packet Filter [13]. On Linux, libpcap uses a memory-mapped `TPACKET_V3` ring that hands *blocks* of packets to user space when a block fills or its timeout expires, trading latency for throughput. Immediate mode [20] removes this delay, but frameworks built on libpcap do not always expose it.

DPI and flow export. nDPI [6] classifies flows and extracts metadata such as the TLS server name. NFStream [2] wraps nDPI in a Python flow-measurement framework that exports a record when a flow ends (idle or active timeout) and runs user plugins on every packet.

Kernel packet filtering. XDP [10] runs verified eBPF programs at the earliest point of the receive path and is used in production for DDoS mitigation [3]. Scholz et al. [18] analysed the performance of eBPF-based filtering, and bpf-iptables [14] showed that rule-by-rule iptables matching degrades with the number of rules while

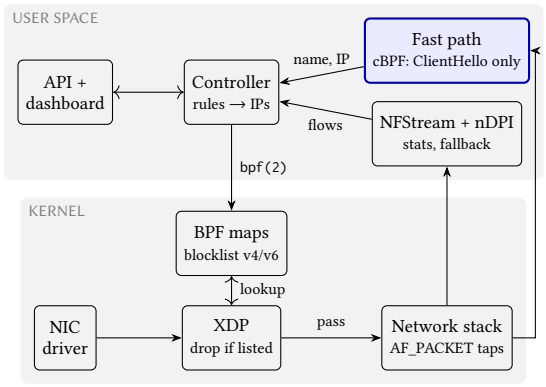


Figure 1: DPI Guard architecture. Two taps observe server names; the controller turns domain rules into IP addresses and writes them to pinned BPF maps that the XDP program consults.

eBPF data structures do not. nftables sets offer hash-based matching inside netfilter, including on the early `netdev` ingress hook.

Name-based blocking and its limits. SNI is sent in clear text in TLS 1.2 and 1.3 [1, 16], which makes it a common signal for filtering and censorship [4]. Encrypted Client Hello (ECH), standardized in 2026 [17, 19], encrypts the whole ClientHello, including the SNI, wherever both ends deploy it. Blocking by IP has collateral effects when domains share addresses, and Hoang et al. [8] show how widely domains are co-hosted.

Why not block at DNS? The simplest domain blocker is a resolver that refuses or rewrites queries for listed names, as with DNS Response Policy Zones [22] or home sinkholes such as Pi-hole. It acts before the first connection exists and costs nothing per packet, but it governs only clients that use that resolver: encrypted DNS such as DoH [9], a hard-coded resolver or a cached or literal address bypasses it. Enforcing on the SNI and server address of the traffic itself does not depend on the client's resolver, which is why DPI Guard pays the cost of turning names into addresses on the data path.

Name-to-address policies with eBPF. The closest prior system is Cilium's DNS-based policy [5]: a DNS proxy observes which addresses a workload's queries resolve to and admits only those, enforced in eBPF. It learns names from DNS answers inside a Kubernetes cluster; DPI Guard learns them from TLS on the wire, so it needs no control over the client's resolver, which brings the time-to-block question we study.

Gap. Prior work evaluates either classification or filtering throughput in isolation. We measure the end-to-end delay between classification and enforcement, and whether a DPI-guided blocker stops the *first* connection.

3 DESIGN

DPI Guard splits the work in two: user space decides *which* addresses to block from the names it observes, and an XDP program in the kernel drops their packets (Figure 1).

Enforcement. One XDP program on the monitored interface parses Ethernet, up to two VLAN tags and the IPv4 or IPv6 header, and looks up both the source and destination address in a hash map with 65,536 entries per address family. A hit returns `XDP_DROP` and increments that entry's packet and byte counters; a miss returns `XDP_PASS`. Matching both directions drops a blocked server's replies on an end host and outbound requests on a gateway. Global counters live in a per-CPU array, so the hot path takes no shared locks.

Decision. Two taps feed the controller. The *fast path* is an `AF_PACKET` socket with a 28-instruction classic-BPF filter that accepts only IPv4 TCP segments to port 443 whose payload begins with a TLS handshake record of type ClientHello; user space therefore wakes only for connection openings, parses the SNI and reports (name, server IP) at once. *NFStream with nDPI* inspects all traffic and keeps per-domain statistics. It is also the fallback for names the fast path cannot read (QUIC, DNS, HTTP, a ClientHello whose server name falls in a later TCP segment): a plugin that runs on every packet reports a name as soon as nDPI has reassembled it. Our first version fell back to flow export instead; §5.3 shows the difference.

Policy. A rule such as `youtube.com` matches the name and all subdomains; the most specific rule wins. The controller remembers every observed (domain, IP) pair, so adding a rule blocks already-seen addresses at once, and removing one unblocks exactly the addresses no remaining rule needs. When a blocked address also serves an allowed name, the controller reports it as collateral.

Implementation. DPI Guard is about 1,000 lines of Python plus a 128-line XDP program compiled ahead of time with clang and loaded with iproute2. The controller manipulates the pinned maps with five `bpf(2)` commands issued through `ctypes`, so it needs no BCC or kernel headers at run time. A Flask API exposes rules and counters. Twenty-six automated tests, three of which run XDP and nftables on a veth pair inside a network namespace, check enforcement end to end.

4 METHODOLOGY

Testbed. A KVM virtual machine with 4 vCPUs (Intel Xeon, 2.8 GHz), 15 GB RAM, Linux 6.18 and iptables 1.8.10 (nf_tables back end). A sender in a network namespace reaches the host through a veth pair; filters attach to the host-side interface (XDP in native mode). A TLS 1.3 server in the namespace answers one HTTP request per connection.

Time-to-block (§5.1). For each configuration we run 100 trials. Trial i connects to its own server address with SNI `t`i`.blocked.test`, so earlier blocks never affect later trials. The client records the time just before sending its ClientHello; DPI Guard records when it wrote the IP to the XDP map. Their difference is the *time-to-block*. Consecutive trials are separated by a random gap of 0.25–1.25 s, so their start times cannot phase-lock with periodic timers in the capture path. A first connection counts as *stopped* if its TLS handshake or HTTP exchange fails within 2 s. Every trial is paired with a control connection to an allowed name on another address, which must succeed. We report medians with 95% percentile-bootstrap confidence intervals and proportions with 95% Wilson intervals.

Emulated round-trip time (§5.2). Our kernel lacks the `netem` queueing discipline, so the server emulates distance itself: it waits D milliseconds after accepting a connection before reading the

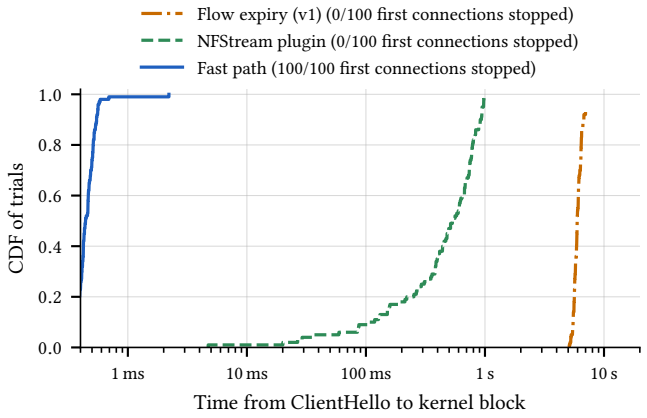


Figure 2: Time from the client's ClientHello to the kernel block, 100 trials per mode. The expiry curve stops at 93/100: the remaining flows were never exported.

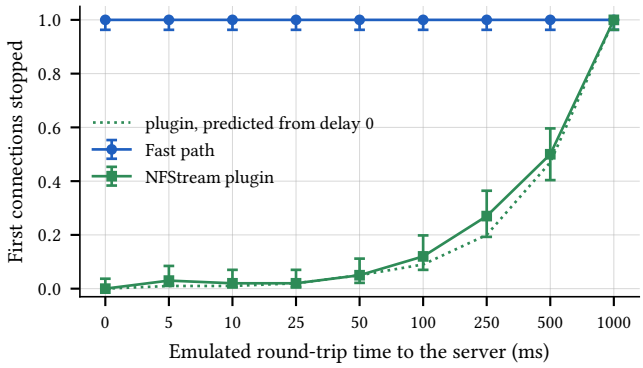


Figure 3: Fraction of first connections stopped against the emulated round-trip time (100 trials per point, 95% Wilson intervals). The dotted line predicts the plugin's curve from its zero-delay time-to-block alone.

Table 1: Time-to-block when the server name arrives in the ClientHello's second TCP segment (median, 95% CI; 100 trials per row).

Fallback	RTT (ms)	Time-to-block	Stopped
Per-packet plugin (v2)	0	466 ms [340 ms, 575 ms]	0/100
Per-packet plugin (v2)	50	522 ms [442 ms, 625 ms]	3/100
Flow export (v1)	0	5.8 s [5.7 s, 6.0 s]	0/100
Flow export (v1)	50	5.9 s [5.8 s, 6.0 s]	0/100

ClientHello and replying. For a blocker next to the client, which is where DPI Guard runs, the server's first reply then arrives D after the ClientHello, as it would with a round-trip time of D. We sweep D from 5 to 1,000 ms.

Split ClientHello (§5.3). Large ClientHellos span several TCP segments; an X25519+ML-KEM-768 hybrid key share alone adds 1,216 bytes. We emulate this by sending the first 100 bytes of the ClientHello in their own segment, which places the server name in the second.

Drop cost and scaling (§5.4). A C sender (`sendmmsg`, batches of 64 UDP packets, 64-byte frames) floods the host for 5 s while pinned to one vCPU. On a veth pair the receive path runs on the sending CPU, so the achieved sender rate measures the whole per-packet path. For drop cost we block the sender's address; for scaling we fill the blocklist with N random addresses that never match, so every packet takes the full no-match path. Each configuration runs 5 times; we report the mean and sample standard deviation.

5 RESULTS

5.1 Time-to-Block

Figure 2 shows the distribution of time-to-block for each mode, with no added delay.

Flow expiry acts only after the connection has ended and stayed idle for the 5 s timeout. Every first connection completed (0/100 stopped), and the block landed a median of 5,940 ms (95% CI 5,863 to 6,061) after the ClientHello, over the flows that were enforced. NFStream checks for idle flows only when new packets arrive: 93 of 100 flows were enforced within the observation window, and the rest, from the last trials before the link went quiet, never were. On a quiet link, export-based enforcement has no upper bound.

The NFStream plugin sees the server name on the connection's fourth packet, yet the block landed 4.73 ms to 995 ms after the ClientHello (median 536 ms, 95% CI 447 to 650). Almost all of this delay lies between a packet's capture timestamp and the plugin call, and it is spread evenly over 0–1 s: the signature of a capture buffer released on a timer. Classification and the map update take well under a millisecond. With no added delay the server replies first, so 0/100 first connections were stopped.

The fast path removes the buffer: the block landed 0.31 ms to 2.21 ms after the ClientHello (median 0.44 ms, 95% CI 0.42 to 0.47; 0.19 ms from frame arrival to map update). The server's first reply was dropped, so 100/100 first connections failed. All 300 control connections succeeded.

5.2 Does the First Connection Survive Distance?

A block stops the first connection only if it lands before the server's reply, so the answer depends on the round-trip time (RTT). Figure 3 sweeps the emulated RTT from 5 to 1,000 ms. The fast path stopped 100/100 first connections at every RTT, including 5 ms. The plugin stopped 5/100 at 50 ms, 12/100 at 100 ms, 50/100 at 500 ms and 100/100 only at 1 s. Its curve follows the prediction from Figure 2, i.e. the fraction of zero-delay trials blocked within the RTT, inside the confidence interval at every point: the plugin's chance of stopping a first connection is set by the capture buffer's timer, not by the network. At the RTTs common on the Internet, tens to a few hundred milliseconds, the plugin lets most first connections through and the fast path lets none through. All 1600 control connections succeeded.

5.3 When the Fast Path Cannot Read the Name

When the server name falls in the ClientHello's second segment, the fast path sees the first segment, finds no name and stays silent; the fallback decides. Table 1 compares our first design, which fell

Table 2: Sender rate with the sender's own address blocked (kpps, 5 runs).

Path	Mean	SD
XDP drop (native)	673	66
nftables `netdev` ingress drop	671	63
iptables `INPUT` drop	582	43
No filter (delivered to socket)	466	35

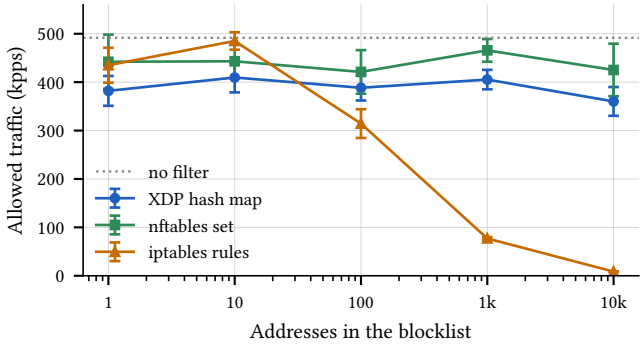


Figure 4: Allowed traffic forwarded while N non-matching addresses are blocked (mean ± SD, 5 runs).

back to flow export, with the current one, which falls back to the per-packet plugin once nDPI has reassembled the ClientHello. The plugin fallback cuts the time-to-block by more than a factor of ten, from about 6 s to about half a second, and enforces every trial; the export fallback missed some flows on the quiet link, as in §5.1. Neither stops many first connections, so ClientHellos that span segments remain the fast path's main gap (§6). All 400 control connections succeeded.

5.4 Drop Cost and Blocklist Scaling

Drop cost. Table 2: dropping is cheaper than delivering, and on veth XDP and the nftables `netdev` ingress hook perform the same within noise, both about 16% faster than an iptables `INPUT` rule, which runs later in the stack. We expected XDP to lead, but on veth the sender has already allocated the socket buffer that XDP normally avoids, and attaching XDP adds a NAPI hand-off. This result shows that the XDP enforcer costs no more than the best netfilter alternative here; its advantage on physical NICs [10, 18] remains to be measured on our hardware.

Scaling. Figure 4: with hash-based lookups the blocklist size does not matter; XDP's map and the nftables set forward allowed traffic at the same rate with 1 or 10,000 entries. iptables evaluates its rules one by one, so every allowed packet pays for every rule: at 10,000 addresses it forwards 8.6 kpps, 2% of its single-rule rate, consistent with the linear degradation reported for iptables [14]. Attaching XDP to a veth interface costs about 20% of forwarding rate even when nothing is dropped, because veth then processes frames through an extra NAPI step; this overhead is specific to veth.

6 DISCUSSION

Where the time goes. In a DPI-guided blocker, the kernel and the classifier are fast; the delay comes from how packets reach the classifier. Export-on-expiry ties enforcement to the flow's end and to later traffic. A per-packet hook fixes the first problem but inherits the capture library's buffering. A narrow, unbuffered tap for the one message that names the server fixes both. Latency-critical decisions in monitoring systems need their own capture path.

An operational lesson. During development, NFStream's meter processes, which ignore `SIGTERM`, outlived the controller and kept capturing; while these orphans ran, blocked traffic intermittently bypassed the XDP program on our veth testbed. The controller now force-kills its children on exit, and every measurement in this paper was taken with that fix in place. Capture processes must be supervised as strictly as the filter itself.

Limitations. *Collateral blocking:* enforcement is per address, so an allowed name sharing a CDN address with a blocked one is blocked too; DPI Guard reports but cannot avoid this [8]. *Hidden names:* ECH is now a standard [17, 19]; where it is deployed, the SNI that DPI Guard (and nDPI) reads is only an innocuous outer name, and encrypted DNS hides the query too. Name-based blocking of any kind loses its signal as ECH spreads. *Large ClientHellos:* this is a common case today, not an edge case. Chrome offers an X25519+ML-KEM-768 hybrid key share by default [7], as do other clients, which pushes the ClientHello past one TCP segment and can place the SNI in the second; middleboxes that read only the first segment have already failed in deployment [15, 21]. DPI Guard's fast path also parses only the first segment, so for such clients enforcement falls back to the plugin's sub-second delay (§5.3). Reassembling the ClientHello in the fast path is the most important next step. *QUIC:* the fast path handles TCP only; QUIC Initial packets are encrypted with keys derived from public values, so a QUIC fast path is feasible but not yet built.

Threats to validity. All measurements come from one virtual machine on a veth pair with sub-millisecond RTT, which hides XDP's main advantage and adds a NAPI step of its own. Round-trip time is emulated by delaying the server's reply, not the network, so it does not model loss or queueing. IPv6 was disabled in the kernel, so only IPv4 was measured. Client and controller timestamps share one clock. Samples are 100 trials per time-to-block configuration and 5 runs per throughput point.

7 CONCLUSION

A DPI-guided blocker is only as fast as the path that carries packets to its classifier. Acting on exported flows took a median of 5,940 ms where it acted at all, and never stopped a first connection; acting inside the DPI engine took 536 ms because of capture buffering, and stopped first connections only as often as that delay predicts; a ClientHello fast path took 0.44 ms and stopped every first connection at every round-trip time, while the XDP blocklist kept a constant cost from 1 to 10,000 entries. Future work: reassembling multi-segment ClientHellos in the fast path, bare-metal runs with native-XDP NICs, collateral measurements on real CDN-hosted domains, a QUIC fast path, and connection-level enforcement to avoid collateral blocking. Code, testbed and data are available [12].

REFERENCES

[1] Donald Eastlake 3rd. 2011. Transport Layer Security (TLS) Extensions: Extension Definitions. RFC 6066.

[2] Zied Aouini and Adrián Pekár. 2022. NFStream: A Flexible Network Data Analysis Framework. *Computer Networks* 204 (2022), 108719.

[3] Gilberto Bertin. 2017. XDP in Practice: Integrating XDP into our DDoS Mitigation Pipeline. In *Proceedings of Netdev 2.1, The Technical Conference on Linux Networking*.

[4] Zimo Chai, Amirhossein Ghafari, and Amir Houmansadr. 2019. On the Importance of Encrypted-SNI (ESNI) to Censorship Circumvention. In *9th USENIX Workshop on Free and Open Communications on the Internet (FOCI)*. USENIX.

[5] Cilium Authors. [n. d.]. Locking Down External Access with DNS-Based Policies. Cilium documentation, https://docs.cilium.io/en/stable/security/dns/. Accessed November 2026.

[6] Luca Deri, Maurizio Martinelli, Tomasz Bujlow, and Alfredo Cardigliano. 2014. nDPI: Open-Source High-Speed Deep Packet Inspection. In *International Wireless Communications and Mobile Computing Conference (IWCMC)*. IEEE, 617–622. https://doi.org/10.1109/IWCMC.2014.6906427

[7] Google Online Security Blog. 2024. A New Path for Kyber on the Web. https://security.googleblog.com/2024/09/a-new-path-for-kyber-on-web.html.

[8] Nguyen Phong Hoang, Arian Akhavan Niaki, Nikita Borisov, Phillipa Gill, and Michalis Polychronakis. 2020. Assessing the Privacy Benefits of Domain Name Encryption. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security (ASIA CCS)*. ACM.

[9] Paul Hoffman and Patrick McManus. 2018. DNS Queries over HTTPS (DoH). RFC 8484.

[10] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. 2018. The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel. In *Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT)*. ACM, 54–66.

[11] Jana Iyengar and Martin Thomson. 2021. QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000.

[12] Shelender Kumar. 2026. DPI Guard: source code, testbed scripts and measurement data. https://github.com/shelenderrkumar/Network-Monitoring-App.

[13] Steven McCanne and Van Jacobson. 1993. The BSD Packet Filter: A New Architecture for User-Level Packet Capture. In *USENIX Winter 1993 Conference*. USENIX.

[14] Sebastiano Miano, Matteo Bertrone, Fulvio Risso, Mauricio Vásquez Bernal, Yunsong Lu, and Jianwen Pi. 2019. Securing Linux with a Faster and Scalable iptables. *ACM SIGCOMM Computer Communication Review* 49, 3 (2019), 3–17. https://doi.org/10.1145/3371927.3371929

[15] MITRE. 2026. CVE-2026-32305: Traefik mTLS Bypass via Fragmented TLS ClientHello. https://www.cve.org/CVERecord?id=CVE-2026-32305.

[16] Eric Rescorla. 2018. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446.

[17] Eric Rescorla, Kazuho Oku, Nick Sullivan, and Christopher A. Wood. 2026. TLS Encrypted Client Hello. RFC 9849.

[18] Dominik Scholz, Daniel Raumer, Paul Emmerich, Alexander Kurtz, Krzysztof Lesiak, and Georg Carle. 2018. Performance Implications of Packet Filtering with Linux eBPF. In *30th International Teletraffic Congress (ITC 30)*. IEEE. https://doi.org/10.1109/ITC30.2018.00039

[19] Benjamin M. Schwartz, Mike Bishop, and Erik Nygren. 2026. Bootstrapping TLS Encrypted ClientHello with DNS Service Bindings. RFC 9848.

[20] The Tcpdump Group. [n. d.]. pcap_set_immediate_mode(3PCAP) manual page. https://www.tcpdump.org/manpages/pcap_set_immediate_mode.3pcap.html. Accessed October 2026.

[21] Traefik Community Forum. 2026. HostSNI Routing Fails on Post-Quantum (ML-KEM) TLS 1.3 Connections due to ClientHello Fragmentation. https://community.traefik.io/t/tcp-router-hostsni-routing-fails-on-post-quantum-ml-kem-tls-1-3-connections-due-to-clienthello-fragmentation/29730.

[22] Paul Vixie and Vernon Schryver. 2017. DNS Response Policy Zones (RPZ). IETF Internet-Draft draft-ietf-dnsop-dns-rpz-00.